קטע קוד A

```
class BoolArray {
  char* arr = nullptr;
  size_t size = 0;
  size_t capacity = 0;

  static size_t actual_size(size_t num_bools) {
    if(num_bools == 0) return 0;
    return (num_bools - 1)/8 + 1;
  }

  void resize(size_t new_size) {
    if(new_size <= capacity) {
      size = new_size;
      return;
    }
    // allocate and initialize all to false
    char* new_arr = new char[actual_size(new_size)] {};
    // memcpy signature: std::memcpy(dst, src, size_in_bytes)
    std::memcpy(new_arr, arr, actual_size(size));
    delete []arr;
    size = new_size;
    capacity = actual_size(new_size) * 8;
    arr = new_arr;
  }

  class BoolProxy {
    BoolArray* const arr;
    size_t index;
    size_t byte_index() const { return index / 8; }
    size_t bit_index() const { return index % 8; }
    char bit_val() const { return 1 << bit_index(); }
  public:
    BoolProxy(BoolArray* arr, size_t index)
        : arr(arr), index(index) {}
    operator bool() const {
      if(index >= arr->size) return false;
      return arr->arr[byte_index()] & bit_val();
    }
    bool operator=(bool value) {
      if(index >= arr->size) arr->resize(index + 1);
      if(value) {
        arr->arr[byte_index()] |= bit_val();
      }
      else {
        arr->arr[byte_index()] &= ~(bit_val());
      }
      return value;
    }
```

```
    bool operator=(const BoolProxy& bp) {
      return *this = static_cast<bool>(bp);
    }
    BoolProxy& operator++() {
      ++index;
      return *this;
    }
    bool operator!=(BoolProxy other) const {
      return arr != other.arr || index != other.index;
    }
  };
public:
  BoolArray() {}
  ~BoolArray() { delete []arr; }
  BoolArray(const BoolArray&) = delete;
  BoolArray& operator=(const BoolArray&) = delete;
  bool operator[](size_t index) const {
    return const_cast<BoolArray&>(*this)[index];
  }
  BoolProxy operator[](size_t index) {
    return BoolProxy{this, index};
  }

  class iterator {
    BoolProxy bp;
  public:
    iterator(BoolProxy bp): bp(bp) {}
    iterator& operator++() {
      ++bp;
      return *this;
    }
    bool operator*() const { return bp; }
    auto& operator*() { return bp; }
    bool operator!=(iterator other) { return bp != other.bp; }
  };
  auto begin() { return iterator{BoolProxy{this, 0}}; }
  auto end() { return iterator{BoolProxy{this, size}}; }
};
```

בעמוד הבא מופיעים מספר main אפשריים לקטע הקוד שלמעלה.

להלן main שסימנו A1

```
int main() {
  BoolArray barr1;
  for(size_t i = 0; i < 10; ++i) {
    if(barr1[i]) barr1[i] = false;
  }
  barr1[15] = false;
}
```

להלן main שסימנו A2

```
int main() {
  cout << std::boolalpha; // print bool as true / false
  BoolArray barr1;
  BoolArray barr2;
  barr2[5] = barr1[5];
  cout << barr2[5] << std::endl;
}
```

להלן main שסימנו A3

```
int main() {
  cout << std::boolalpha; // print bool as true / false
  BoolArray barr3;
  // barr3[3] = true;
  std::cout << barr3[3] << std::endl;
  bool val = true;
  for(auto& b : barr3) {
    b = val;
    val = !val;
  }
  for(auto b : barr3) {
    std::cout << b << std::endl;
  }
}
```

קטע קוד B

```
class Expression {
public:
  Expression() = default;
  Expression(const Expression&) = delete;
  Expression& operator=(const Expression&) = delete;
  virtual ~Expression() {}
  virtual double eval()const = 0;
  virtual void print(ostream& out)const = 0;
  friend ostream& operator<<(ostream& out, const Expression& e) {
    e.print(out);
    return out;
  }
};

class BinaryExpression: public Expression {
  Expression *_e1, *_e2;
  char _sign;
  virtual double eval(double d1, double d2)const = 0;
public:
  BinaryExpression(Expression *e1, Expression *e2, char sign)
    : _e1(e1), _e2(e2), _sign(sign) {}
  ~BinaryExpression() override {delete _e1; delete _e2;}
  double eval()const override {
    return eval(_e1->eval(), _e2->eval());
  }
  void print(ostream& out)const override {
    out << '(' << *_e1 << _sign << *_e2 << ')';
  }
};

class Sum: public BinaryExpression {
  double eval(double d1, double d2)const override {
    return d1 + d2;
  }
public:
  Sum(Expression *e1, Expression *e2)
    : BinaryExpression(e1, e2, '+') {}
};

class Exponent: public BinaryExpression {
  double eval(double d1, double d2)const override {
    return std::pow(d1, d2);
  }
public:
  Exponent(Expression *e1, Expression *e2)
    : BinaryExpression(e1, e2, '^') {}
};
```

```
48. class Number: public Expression {
49.   double _d;
50. public:
51.   Number(double d) : _d(d) {}
52.   double eval()const override {
53.     return _d;
54.   }
55.   void print(ostream& out)const override {
56.     out << _d;
57.   }
58. };
59.
60. int main() {
61.     Expression* e = new Sum(
62.         new Exponent(
63.             new Number(2),
64.             new Number(3)),
65.         new Number(-2));
66.
67.     cout << *e << " = " << e->eval() << endl;
68.     delete e;
69. }
```

קטע קוד C – שאלות בונוס

```
 1. template<class T>
 2. class Stack {
 3.   std::vector<T> stack;
 4. public:
 5.   Stack& push(const T& t) {
 6.     stack.push_back(t);
 7.     return *this;
 8.   }
 9.   Stack& push(T&& t) {
10.     stack.push_back(std::move(t));
11.     return *this;
12.   }
13.   T pop() {
14.     T t = std::move(stack.back());
15.     stack.pop_back(); // remove the last element
16.     return t;
17.   }
18.   size_t size() const { return stack.size(); }
19. };
20.
21. class Foo {
22.   std::string s;
23. public:
24.   Foo(const char* s = "@"): s(s) {}
25.   void print() const { std::cout << s << std::endl; }
26. };
27.
28. int main() {
29.   Stack<std::string> s1;
30.   s1.push("hi").push(std::string{"bye"});
31.   std::cout << s1.pop() << std::endl;
32.
33.   // auto s2 = std::move(s1);
34.   // s2.push("hello");
35.   // std::cout << s2.size() << std::endl;
36.
37.   // Stack<Foo> s3;
38.   // Foo f;
39.   // s3.push(f);
40.   // f.print();
41. }
```

בהצלחה !